

SQL RDBMS Best Practices and Industry Standards for Developers

1. Database Design Best Practices

a. Normalization & Denormalization

- Normalize data up to **3rd Normal Form (3NF)** to reduce redundancy.
- Denormalize selectively for performance optimization in read-heavy databases.

b. Proper Data Types

- Use appropriate **data types** (e.g., `INT` for numbers, `VARCHAR(n)` for text).
- Avoid using `TEXT` or `BLOB` unless necessary.

c. Use Primary Keys (PK) & Foreign Keys (FK)

- Ensure **every table has a primary key** (preferably `AUTO_INCREMENT` for IDs).
- Establish **foreign keys** for referential integrity.

d. Indexing Strategies

- Use **indexes** on frequently queried columns (e.g., `WHERE`, `JOIN` conditions).
- Avoid excessive indexing—each index **slows down inserts/updates**.
- Prefer **composite indexes** for multi-column searches.

e. Use Constraints & Defaults

- Use `UNIQUE`, `CHECK`, and `DEFAULT` constraints for data integrity.
 - Avoid `NULL` values where possible, as they complicate queries.
-

2. Query Optimization Best Practices

a. Write Efficient Queries

- Use `EXPLAIN` to analyze query execution plans.
- Prefer `JOIN` over **subqueries** for better performance.

- Fetch only necessary columns instead of `SELECT *`.

b. Use Stored Procedures & Prepared Statements

- Improve performance and security by **using stored procedures** for frequently used queries.
- Use **prepared statements** to prevent SQL injection and improve execution speed.

c. Optimize WHERE Clause

- Place indexed columns in **WHERE conditions**.
- Avoid functions on indexed columns (e.g., `WHERE LOWER(name) = 'john'` is bad for indexing).

d. Pagination Best Practices

- Use `LIMIT` and `OFFSET` for large result sets.
 - Consider keyset pagination (`WHERE id > last_seen_id LIMIT 50`) for performance.
-

3. Performance Optimization Techniques

a. Use Connection Pooling

- Reuse database connections to **reduce overhead**.
- Configure connection pools properly based on application load.

b. Optimize Transactions

- Keep transactions **short and fast** to avoid locking issues.
- Use `COMMIT` and `ROLLBACK` appropriately.

c. Partitioning & Sharding

- Use **partitioning** for very large tables to improve query performance.
- Consider **sharding** for horizontal scaling.

d. Caching Strategies

- Use **query caching** for frequently accessed data.
- Implement caching layers like **Redis or Memcached**.

4. Security Best Practices

a. Prevent SQL Injection

- Always use **parameterized queries** or **prepared statements**.
- Avoid dynamic SQL unless absolutely necessary.

b. Use Proper User Roles & Permissions

- Grant **least privilege access** (e.g., **READ-ONLY** for non-admin users).
- Avoid using **root** or **admin** for application queries.

c. Encrypt Sensitive Data

- Use **encryption** for storing passwords (**bcrypt**, **argon2**).
- Use **SSL/TLS** for database connections.

5. Backup & Disaster Recovery

a. Regular Backups

- Schedule **automated backups** (full, incremental, differential).
- Store backups securely offsite.

b. Monitor Database Health

- Use tools like **Prometheus**, **Grafana**, or **built-in RDBMS monitoring**.
- Track query performance, slow logs, and database errors.

c. Replication & Failover

- Use **replication** for high availability (e.g., MySQL Replication, PostgreSQL Streaming Replication).
 - Implement automatic **failover** with clustering solutions.
-

6. Industry Standards & Compliance

a. Follow ACID Principles

- Ensure **Atomicity, Consistency, Isolation, and Durability (ACID)**.

b. Compliance with Regulations

- Follow **GDPR, HIPAA, or PCI-DSS** for data security.
- Store logs for auditing and compliance verification.

c. Documentation & Version Control

- Maintain **ER diagrams and schema documentation**.
 - Use **Liquibase or Flyway** for database migrations.
-

Normalization in SQL

1. First Normal Form (1NF)

Goal: Eliminate repeating groups; ensure atomic values.

Rules:

- Each column should contain atomic (indivisible) values.
- Each column must contain values of the same type.
- Each row should be unique.

Example (Before 1NF):

StudentID	Name	Subjects
1	Arjun	Math, Science
2	Priya	English, History

After 1NF:

StudentID	Name	Subject
1	Arjun	Math
1	Arjun	Science
2	Priya	English
2	Priya	History

2. Second Normal Form (2NF)

Goal: Remove partial dependencies (i.e., fields depending only on part of a composite key).

Rules:

- Must be in 1NF.
- All non-key attributes must be fully functionally dependent on the primary key.

Use case: Only relevant when the table has a composite primary key.

3. Third Normal Form (3NF)

Goal: Remove transitive dependencies.

Rules:

- Must be in 2NF.
- Non-key columns must not depend on other non-key columns.

Example (Before 3NF):

EmpID	EmpName	DeptID	DeptName
1	Ravi	10	HR

Here, **DeptName** depends on **DeptID**, not directly on **EmpID**.

After 3NF:

- Separate department info into another table.
-

4. Boyce-Codd Normal Form (BCNF)

Goal: Handle edge cases not covered by 3NF.

Rule:

- Every determinant must be a candidate key.

Slightly stronger version of 3NF.

5. Fourth Normal Form (4NF)

Goal: Remove multi-valued dependencies.

Rules:

- Must be in BCNF.
 - A record should not contain two or more independent multi-valued facts about an entity.
-

Summary Table:

Normal Form	Key Feature
1NF	Atomic columns, no repeating groups
2NF	No partial dependency
3NF	No transitive dependency
BCNF	Every determinant is a candidate key
4NF	No multi-valued dependency

Types of Keys in SQL

Keys are used to uniquely identify rows in a table and maintain relationships between tables.

1. Primary Key

- Uniquely identifies each record.
- Cannot be **NULL**.
- Only one primary key per table.

```
CREATE TABLE Students (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

2. Foreign Key

- Creates a relationship between two tables.
- Points to the primary key in another table.

```
CREATE TABLE Enrollments (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)  
);
```

3. Candidate Key

- A column (or set of columns) that can qualify as a primary key.
 - There can be multiple candidate keys, but only one is chosen as the primary key.
-

4. Composite Key

- A primary key made of two or more columns.

```
CREATE TABLE Orders (  
    OrderID INT,  
    ProductID INT,  
    PRIMARY KEY (OrderID, ProductID)  
);
```

5. Unique Key

- Ensures all values in a column (or group of columns) are unique.
 - Can have multiple unique keys.
 - Allows one `NULL`.
-

Types of Joins in SQL

Joins are used to fetch data from multiple tables based on a related column.

1. INNER JOIN

- Returns matching records from both tables.

```
SELECT * FROM Students  
INNER JOIN Enrollments ON Students.StudentID = Enrollments.StudentID;
```

2. LEFT JOIN (LEFT OUTER JOIN)

- Returns all records from the **left** table and matched records from the right.

```
SELECT * FROM Students  
LEFT JOIN Enrollments ON Students.StudentID = Enrollments.StudentID;
```

3. RIGHT JOIN (RIGHT OUTER JOIN)

- Opposite of LEFT JOIN. Returns all from the **right**, and matched from the left.

```
SELECT * FROM Enrollments  
RIGHT JOIN Students ON Students.StudentID = Enrollments.StudentID;
```

4. FULL JOIN (FULL OUTER JOIN)

- Returns all records when there is a match in either table.

```
SELECT * FROM Students  
FULL OUTER JOIN Enrollments ON Students.StudentID =  
Enrollments.StudentID;
```

5. CROSS JOIN

- Returns Cartesian product (all combinations).

```
SELECT * FROM Products  
CROSS JOIN Categories;
```

6. SELF JOIN

- Table joins with itself.

```
SELECT A.Name AS Employee, B.Name AS Manager  
FROM Employees A  
JOIN Employees B ON A.ManagerID = B.EmployeeID;
```



1. DDL – Data Definition Language

Used to define and modify **structure** of database objects like tables.

Command	Purpose
CREATE	Creates a new table, view, or database.
ALTER	Modifies an existing table (add/delete columns, etc.).
DROP	Deletes a table or database.
TRUNCATE	Removes all records from a table (cannot be rolled back).

Example:

```
CREATE TABLE Students (  
    ID INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

```
ALTER TABLE Students ADD Age INT;
```

```
DROP TABLE Students;
```



2. DML – Data Manipulation Language

Used to manage **data inside tables**.

Command	Purpose
INSERT	Adds new records.
UPDATE	Modifies existing records.
DELETE	Removes specific records.

Example:

```
INSERT INTO Students (ID, Name, Age) VALUES (1, 'Arjun', 20);
```

```
UPDATE Students SET Age = 21 WHERE ID = 1;
```

```
DELETE FROM Students WHERE ID = 1;
```

3. DQL – Data Query Language

Used for **querying data**.

Command	Purpose
SELECT	Retrieves data from one or more tables.

Example:

```
SELECT * FROM Students;
```

```
SELECT Name, Age FROM Students WHERE Age > 18;
```

4. DCL – Data Control Language

Used to control **access/permissions**.

Command	Purpose
GRANT	Gives user access privileges.
REVOKE	Removes user privileges.

Example:

```
GRANT SELECT ON Students TO user_name;
```

```
REVOKE SELECT ON Students FROM user_name;
```



5. TCL – Transaction Control Language

Used to manage **transactions** in a database.

Command	Purpose
COMMIT	Saves all changes made.
ROLLBACK	Undoes changes since last commit.
SAVEPOINT	Sets a save point within a transaction.

Example:

```
BEGIN;  
UPDATE Students SET Age = 25 WHERE ID = 2;  
SAVEPOINT sp1;  
UPDATE Students SET Age = 30 WHERE ID = 3;  
ROLLBACK TO sp1;  
COMMIT;
```



Types of Complex Subqueries

1. Subquery in **WHERE** clause

Used to filter based on the result of another query.



Example: Get patients who have appointments with a *cardiologist*

```
SELECT Name FROM Patients  
WHERE PatientID IN (  
    SELECT PatientID FROM Appointments  
    WHERE DoctorID IN (  
        SELECT DoctorID FROM Doctors  
        WHERE Specialization = 'Cardiology'  
    )  
);
```

2. Subquery in **FROM** clause

Often used to create a temporary table or derived dataset.

Example: Count of appointments per doctor, then get doctors with more than 5 appointments

```
SELECT DocName, TotalAppointments
FROM (
    SELECT D.Name AS DocName, COUNT(*) AS TotalAppointments
    FROM Appointments A
    JOIN Doctors D ON A.DoctorID = D.DoctorID
    GROUP BY D.Name
) AS DoctorCounts
WHERE TotalAppointments > 5;
```

3. Subquery in **SELECT** clause

Calculates extra data for each row.

Example: List each patient with their total number of appointments

```
SELECT Name,
    (SELECT COUNT(*) FROM Appointments WHERE PatientID = P.PatientID)
AS TotalAppointments
FROM Patients P;
```

4. Correlated Subquery

This is like a subquery that *depends* on the outer query for each row.

Example: List doctors who have more than the average number of appointments

```
SELECT Name FROM Doctors D
WHERE (
    SELECT COUNT(*) FROM Appointments A WHERE A.DoctorID = D.DoctorID
) > (
    SELECT AVG(AppointmentCount) FROM (
```

```
        SELECT COUNT(*) AS AppointmentCount FROM Appointments GROUP BY
DoctorID
    ) AS AvgCounts
);
```

5. Using **EXISTS**

Efficient way to check presence of related rows.

Example: Get doctors who have at least one appointment

```
SELECT Name FROM Doctors D
WHERE EXISTS (
    SELECT 1 FROM Appointments A WHERE A.DoctorID = D.DoctorID
);
```

Tips for Complex Subqueries:

- Use **aliases** for clarity.
- Subqueries can slow things down — consider using **JOIN** if performance matters.
- Use **EXISTS** for faster performance compared to **IN** for larger datasets.

Advanced SQL Concepts

1. Window Functions (a.k.a. Analytical Functions)

They perform calculations **across a set of table rows** related to the current row.

Example: Get each patient's total appointments and their rank

```
SELECT
    PatientID,
    COUNT(*) OVER (PARTITION BY PatientID) AS TotalAppointments,
    RANK() OVER (ORDER BY COUNT(*) OVER (PARTITION BY PatientID) DESC)
AS Rank
FROM Appointments;
```

 Functions: `RANK()`, `DENSE_RANK()`, `ROW_NUMBER()`, `SUM()`, `AVG()` with `OVER()` clause

2. Common Table Expressions (CTE)

Temporary result set used within a `SELECT`, `INSERT`, `UPDATE`, or `DELETE`.

Example: Get doctors with more than 5 appointments

```
WITH DoctorCount AS (
    SELECT DoctorID, COUNT(*) AS TotalAppointments
    FROM Appointments
    GROUP BY DoctorID
)
SELECT * FROM DoctorCount WHERE TotalAppointments > 5;
```

 Makes queries modular, readable, and reusable.

3. Recursive CTEs

Used for hierarchical or tree structures.

Example: Employees and managers (basic structure)

```
WITH RECURSIVE EmployeeHierarchy AS (  
    SELECT EmployeeID, Name, ManagerID  
    FROM Employees  
    WHERE ManagerID IS NULL  
    UNION ALL  
    SELECT E.EmployeeID, E.Name, E.ManagerID  
    FROM Employees E  
    JOIN EmployeeHierarchy EH ON E.ManagerID = EH.EmployeeID  
)  
SELECT * FROM EmployeeHierarchy;
```

4. JSON in SQL (Modern DBs like PostgreSQL, MySQL, SQL Server)

Can store and query **JSON data** directly inside SQL.

```
SELECT PatientData->>'Name' AS Name  
FROM Patients  
WHERE PatientData->'MedicalHistory'->>'Diabetes' = 'Yes';
```

5. Pivoting and Unpivoting

Used to **rotate** rows to columns and vice versa.

Example (simplified pivot):

```
SELECT PatientID,  
    MAX(CASE WHEN TestName = 'Blood' THEN Result END) AS BloodResult,  
    MAX(CASE WHEN TestName = 'X-Ray' THEN Result END) AS XrayResult  
FROM TestResults  
GROUP BY PatientID;
```



6. Indexing & Performance Tuning

- **Indexes** improve SELECT performance but can slow down INSERT/UPDATE.
 - Use **EXPLAIN** or **QUERY PLAN** to analyze performance.
 - Optimize using **indexes**, **proper joins**, **less subqueries**, etc.
-



7. Temporary Tables & Global Temp Tables

Used for storing intermediate results.

```
CREATE TEMP TABLE TempPatients AS  
SELECT * FROM Patients WHERE Age > 60;
```



8. Transactions & Isolation Levels

Used for **data consistency**.

```
BEGIN TRANSACTION;
```

```
UPDATE Accounts SET Balance = Balance - 100 WHERE ID = 1;  
UPDATE Accounts SET Balance = Balance + 100 WHERE ID = 2;
```

```
COMMIT;
```

Isolation levels: **READ UNCOMMITTED**, **READ COMMITTED**, **REPEATABLE READ**,
SERIALIZABLE



9. Data Integrity Constraints

- **CHECK:** `CHECK (Age > 0)`
 - **DEFAULT:** `Gender VARCHAR(10) DEFAULT 'Unknown'`
 - **NOT NULL, UNIQUE, PRIMARY/FOREIGN KEY**
-



10. Materialized Views

Precomputed views stored on disk for heavy queries.

```
CREATE MATERIALIZED VIEW TopDoctors AS
SELECT DoctorID, COUNT(*) AS TotalAppointments
FROM Appointments
GROUP BY DoctorID;
```



What is JSON in SQL?

Modern databases like **PostgreSQL**, **MySQL 5.7+**, and **SQL Server 2016+** support storing and querying JSON data directly.

You can:

- Store a full JSON object in a column
- Extract data using keys
- Filter rows based on JSON values
- Modify JSON fields

SQL Server (2016 and later) supports **JSON natively**, but in its own style. Let's walk through how to **store**, **read**, **filter**, and **update** JSON in SQL Server, using our **hospital app example**.



1. Creating a Table with JSON Column

```
CREATE TABLE Patients (  
    PatientID INT PRIMARY KEY,  
    Name NVARCHAR(100),  
    MedicalRecord NVARCHAR(MAX)  -- Store JSON as plain NVARCHAR  
);
```

💡 SQL Server doesn't have a dedicated **JSON** datatype — it stores JSON as text but provides built-in functions to work with it.



Sample Data (Insert JSON)

```
INSERT INTO Patients (PatientID, Name, MedicalRecord)  
VALUES (  
    1, 'Drishti R.',  
    '{  
        "blood_group": "O+",&br/>        "allergies": ["pollen", "dust"],  
        "conditions": {  
            "diabetes": true,  
            "asthma": false  
        }  
    }'  
);
```



2. Reading JSON Data



Get Blood Group

```
SELECT  
    Name,  
    JSON_VALUE(MedicalRecord, '$.blood_group') AS BloodGroup  
FROM Patients;
```



Check if patient has diabetes

```
SELECT Name
FROM Patients
WHERE JSON_VALUE(MedicalRecord, '$.conditions.diabetes') = 'true';
```

3. Extracting Complex JSON – OPENJSON

List all allergies from the array:

```
SELECT value AS Allergy
FROM Patients
CROSS APPLY OPENJSON(MedicalRecord, '$.allergies')
WHERE PatientID = 1;
```

4. Modifying JSON

SQL Server doesn't allow direct JSON modification, so you use `JSON_MODIFY`.

Change blood group to 'A+':

```
UPDATE Patients
SET MedicalRecord = JSON_MODIFY(MedicalRecord, '$.blood_group', 'A+')
WHERE PatientID = 1;
```

Add a new field (e.g., smoker status):

```
UPDATE Patients
SET MedicalRecord = JSON_MODIFY(MedicalRecord, '$.smoker', 'false')
WHERE PatientID = 1;
```

5. Validating JSON

```
SELECT *
FROM Patients
WHERE ISJSON(MedicalRecord) = 1;
```

6. Query JSON with **OPENJSON** into Table Format

```
SELECT *
FROM OPENJSON('{
    "blood_group": "B+",
    "diabetes": true,
    "age": 40
}')
```

```
WITH (
    blood_group NVARCHAR(10),
    diabetes BIT,
    age INT
);
```

This is super handy when you need to parse incoming JSON from an API or app.

Use Cases in Hospital App

Use Case	JSON Benefit
Medical history	Dynamic, varies by patient
Allergy details	Arrays inside JSON
Scans/reports metadata	Store image info (file, date, etc.)
Emergency contact details	Optional nested info

When to Use JSON in SQL

- Patient histories (unstructured, varies person to person)
- Medical notes / scan metadata
- App logs or audit trails

 **Avoid JSON** for structured, relational data (like **DoctorID**, **AppointmentDate**, etc.) — it's slower to query and harder to index.

Performance Tuning Territory — Indexing in SQL Server

Is **crucial** when your database starts growing and queries slow down. Think of indexes like the index page of a book — they help SQL Server *find rows faster*, rather than reading the whole table.

What Is an Index?

An **index** is a data structure (usually B-tree) that SQL Server uses to quickly look up data **based on column values** — especially in **WHERE**, **JOIN**, **ORDER BY**, and **GROUP BY** clauses.

Types of Indexes in SQL Server

1. Clustered Index

- Only **one per table**.
- Sorts and stores the actual data **physically** in that order.
- By default, **PRIMARY KEY** creates a clustered index.

```
CREATE CLUSTERED INDEX IX_Patients_ID  
ON Patients (PatientID);
```

2. Non-Clustered Index

- Creates a **separate structure** with pointers to the original data.
- You can have **multiple non-clustered indexes** per table.

```
CREATE NONCLUSTERED INDEX IX_Patients_Name  
ON Patients (Name);
```



Hospital App Indexing Examples

Let's say we have this table:

```
CREATE TABLE Appointments (  
    AppointmentID INT PRIMARY KEY,          -- Clustered by default  
    PatientID INT,  
    DoctorID INT,  
    AppointmentDate DATETIME,  
    Status VARCHAR(20)  
);
```

⚡ Speed up queries like:

-- Find appointments by doctor and date

```
SELECT * FROM Appointments  
  
WHERE DoctorID = 1001 AND AppointmentDate > '2025-01-01';
```

👉 Add a non-clustered index:

```
CREATE NONCLUSTERED INDEX IX_DoctorDate  
  
ON Appointments (DoctorID, AppointmentDate);
```

How to Analyze Performance

SET STATISTICS IO ON and SET STATISTICS TIME ON

These help you see reads & execution time.

Execution Plan

Use **SQL Server Management Studio (SSMS)**:

- Click **Include Actual Execution Plan** before running a query.
 - Hover over the steps to see **which indexes were used**, and **which were missing**.
-



Composite Indexes

Indexes on multiple columns (order matters!)

```
CREATE NONCLUSTERED INDEX IX_Patient_Status  
ON Appointments (PatientID, Status);
```

Query:

```
SELECT * FROM Appointments WHERE PatientID = 501 AND Status =  
'Completed';
```

💡 WHERE Status = 'Completed' AND PatientID = 501 might **not** use the index efficiently, because SQL Server uses the **leftmost column** of the index first.

✓ Best Practices

Rule	Tip
✓	Index columns used in WHERE , JOIN , ORDER BY , GROUP BY
✗	Don't over-index — every insert/update/delete slows down
✓	Use included columns to cover queries without looking up table
✓	Use filtered indexes for sparse data (e.g., only Status = 'Active')

📌 Included Column Example:

```
CREATE NONCLUSTERED INDEX IX_AppointmentDate  
ON Appointments (AppointmentDate)  
INCLUDE (DoctorID, Status);
```

Covers this query:

```
SELECT AppointmentDate, DoctorID, Status  
FROM Appointments  
WHERE AppointmentDate > '2025-01-01';
```
